# 3.1.1 DBT - DBT Blueprint

Last updated by | Marta Pereira | Feb 26, 2026 at 11:45 PM GMT

Contents


- DBT Blueprint
  - Convert the Blueprint into a valid DBT Project
  - DBT Project Structure
  - Project File
  - Profiles File
  - Macros
  - Models
    - Sources File
    - Model File
    - Schema File
  - Seeds
  - Tests


## DBT Blueprint

This blueprint provides a reusable DBT project template designed to run on Databricks. It can be adapted for various use cases by replacing placeholders and adjusting configuration details as needed. It can be found the **Databricks-DBT-Blueprint** folder within the **dbt-template** repository, while a functioning project that makes use of the blueprint with some extra additions can be found in the **Databricks-DBT-Example** folder.

This project is based on a three-layer data architecture: **bronze**, **silver**, and **gold**. The **bronze** layer is populated exclusively through data ingestion, while DBT is used to transform that data into the **silver** and **gold** layers.

### Convert the Blueprint into a valid DBT Project

To convert the blueprint you must:

1. Create the DBT profile in the **profiles.yml** file. (How-To)
2. Decided on the DBT project configurations in the **dbt_project.yml** files. (How-To)
3. Fill in the correct source tables for the bronze layer in the **models/bronze/sources.yml** file. (How-To)
4. Create a silver layer model in the **models/silver/silver_model.sql** file. (How-To)
5. Fill in the schema for the newly created model in the **models/silver/schema.yml** file. (How-To)
6. Add any seeds you want and fill the **seeds/properties.yml** file. (How-To)
7. Run DBT following the Run DBT page.

### DBT Project Structure

The following structure defines a valid DBT project, with support for models, macros, seeds, and tests:

```
Databricks-DBT-Blueprint
│   dbt_project.yml
│   profiles.yml
│
├───macros
│   │   generate_schema_name.sql
│
├───models
│   ├───bronze
│   │   │   sources.yml
│   │
│   ├───silver
│   │   │   schema.yml
│   │   │   silver_model.sql
│   │
│   └───gold
│
├───seeds
│   │   properties.yml
│
└───tests
    └───generic
```

## Project File

The **dbt_project.yml** file defines core project configurations and must be placed at the project root. See the [dbt_project.yml reference]() for more details.

```yaml
name: '<project-name>'

config-version: 2
version: '<current-project-version>'

profile: '<profile-name>'

model-paths: ["models"]
seed-paths: ["seeds"]
test-paths: ["tests"]

require-dbt-version: [">=1.0.0", "<2.0.0"]

models:
  <project-name>:
    silver:
      +database: <silver-default-target-catalog>
      +schema: <silver-default-target-schema>
      +materialized: <silver-default-materialization-type>
    gold:
      +database: <gold-default-target-catalog>
      +schema: <gold-default-target-schema>
      +materialized: <gold-default-materialization-type>

tests:
  +store_failures: <store-failures>
```

Placeholders:

- **project-name**: Name of the DBT project.
- **current-project-version**: Version of the current project.
- **profile-name**: DBT profile to use **(must match one in profiles.yml)**.
- **{layer}-default-target-catalog**: Default catalog where the models for each layer (silver/gold) will be stored. (Optional)

- **{layer}-default-target-schema**: Default schema where the models for each layer (silver/gold) will be stored. (Optional)
- **{layer}-default-materialization-type**: Default materialization strategy (table, view, incremental, ephemeral, or materialized view). **Default is view**. (Optional)
- **store-failures**: Whether to store test results or not. If set to true, it will create a **'dbt_test__audit'** schema for every single test, whether they fail or not. (Optional - Default is false).

**Note**: DBT uses the default schema passed in the **profiles.yml** file and what's added to the **+schema** property will cause the target schema to be created as **'<profiles-schema>_<+schema>'**. To override this behavior and have the target schema as the value in the **+schema** only, this project has a defined macro in **macros/generate_schema_name.sql**. **If you don't want this behaviour, delete the macro.**

## Profiles File

The **profiles.yml** file stores connection details for the target platform. Reference structure and more info can be found in the [profiles.yml documentation]() .

Below is an example profile supporting Service Principal authentication method for a Databricks All Purpose Compute cluster:

```yaml
<profile-name>:
  target: dev
  outputs:
    dev:
      type: databricks
      method: http
      host: <dev-host>
      http_path: <dev-http-path>
      catalog: <dev-catalog>
      schema: <dev-schema>
      threads: 1
      auth_type: oauth
      azure_client_id: "{{ env_var('CLIENT_ID') }}"
      azure_client_secret: "{{ env_var('CLIENT_SECRET') }}"
    qa:
      type: databricks
      method: http
      host: <qa-host>
      http_path: <qa-http-path>
      catalog: <qa-catalog>
      schema: <qa-schema>
      auth_type: oauth
      azure_client_id: "{{ env_var('CLIENT_ID') }}"
      azure_client_secret: "{{ env_var('CLIENT_SECRET') }}"
    prod:
      type: databricks
      method: http
      host: <prod-host>
      http_path: <prod-http-path>
      catalog: <prod-catalog>
      schema: <prod-schema>
      threads: 1
      auth_type: oauth
      azure_client_id: "{{ env_var('CLIENT_ID') }}"
      azure_client_secret: "{{ env_var('CLIENT_SECRET') }}"
```

Placeholders:

- **profile-name**: Name of the DBT profile.
- **{env}-host**: Databricks SQL Warehouse or All Purpose Compute Server Hostname.
- **{env}-http-path**: Databricks SQL Warehouse or All Purpose Compute HTTP Path.

- **{env}-catalog**: Databricks Catalog.
- **{env}-schema**: Default Databricks Schema.

**Notes**:

- The **client-id**, and **client-secret** properties are expecting an environmental variable with their values for security purposes.
- To run DBT with a different target than the default (**dev**), run: `dbt run --target <target-name>`
- **SQL Warehouses must be prioritized over All Purpose Compute as defined in [Architecture Decisions - DBT Approach](#).**

## Macros

The macros folder includes any re-usable piece of SQL code necessary for the project, of which more information can be found in the Jinja and macros page . In this blueprint, this macro is defined (refer to the Project File notes for its usage):

```
{% macro generate_schema_name(custom_schema_name, node) -%}

    {%- set default_schema = target.schema -%}
    {%- if custom_schema_name is none -%}

        {{ default_schema }}

    {%- else -%}

        {{ custom_schema_name }}

    {%- endif -%}

{%- endmacro %}
```

**Note**: There is also a commented **generate_database_name.sql** macro in the blueprint, which, when in use, overrides the way DBT obtains the catalog name and adds the currently used **target** defined in the **profiles.yml** file as a suffix. For example, it transforms the catalog **'catalog_bronze'** with the target **'dev'** to **'catalog_bronze_dev'**. It's useful to have the project work regardless of its environment.

## Models

Models define SQL logic that transforms data. Place them inside the **models** folder, and organize by layers (**silver**, **gold**, etc.).

### Sources File

In the model queries, any number of table sources can be used by declaring them in a **sources.yml** file:

```
version: 2

sources:
  - name: <source-name>
    database: "{{ target.catalog }}"
    schema: <schema-name>
    tables:
      - name: <table-name>
      - name: <table-name>
```

Placeholders:

- **source-name**: Source name.
- **schema-name**: Schema name.
- **table-name**: Table name.

**Notes**:

- To reference these tables use `source('<source-name>', '<table-name>'` . For internal DBT models, use: `ref('<model-name>')` , without the need for them to be declared in a **sources.yml** file.
- The `"{{ target.catalog }}"` will use the catalog defined in the target profile in the **profiles.yml** file to allow for a seamless deployment between environments that will use similar schemas, but different catalogs.

### Model File

Each model must be defined in its own SQL file and it must feature a SQL query which returns a tabular result. The following is a basic model file that only loads data from a source:

```
{{ config(
    materialized="<materialization-type>",
    schema="<schema>",
    post_hook=[
        "OPTIMIZE {{ this }} ZORDER BY (<column-name>)",
        "VACUUM {{ this }} RETAIN 168 HOURS"
    ]
) }}

select
    *
from {{ source('<source-name>', '<source-table>') }}
```

Placeholders:

- **materialization-type**: Materialization type. **Overrides any other materialization strategy previously defined.** (Optional)
- **schema**: Target schema. **Overrides any other target schema previously defined**. (Optional)
- **column-name**: The column(s) to use for ZORDER. **Hooks are optional.**
- **source-name**: Source name for a source defined in a **sources.yml** file.
- **source-table**: Source table for a source defined in a **sources.yml** file.

**Note:** There is support for Python models , which can be used for operations that are difficult or cannot be done with SQL. **They only support 'table' and 'incremental' materializations and they cannot be run on a SQL warehouse. Python Models cannot have runtime variables in the their hooks, meaning 'VACUUM' and 'OPTIMIZE' hooks which require selecting its own table can't be used due to changes in table paths between environments.**

### Schema File

A schema can also be defined for each table in a **schema.yml** file, which also allows for other configurations of the model, including adding an alias and tests:

```yaml
version: 2

models:
  - name: <model-name>
    description: <model-description>
    columns:
      - name: <column-name>
        description: <column-description>
        tests:
          - <test-name>
          - <test-name>
      - name: <column-name>
        description: <column-description>
        tests:
          - <test-name>
```

Placeholders:

- **model-name**: Model name.
- **model-description**: Model description. (Optional)
- **column-name**: Column name.
- **column-description**: Column description. (Optional)
- **test-name**: Test name. Either existing or defined DBT generic tests. (Check the Tests section for more information) (Optional)

### Seeds

Seeds allow you to load CSV files into your database for testing. Each CSV placed in the **seeds** directory is turned into a table with the same name using the `dbt seed` command. You can defined a **properties.yml** file to have better control on how the seeds are created. You can check the Seed configurations page for more information.

```yaml
version: 2

seeds:
  - name: <seed-name>
    config:
      column_types:
        <column1-name>: <column1-data-type>
        <column2-name>: <column2-data-type>
        <column3-name>: <column3-data-type>
```

Placeholders:

- **seed-name**: Seed name. The CSV file's name without the extension.
- **column{n}-name**: Column name.
- **column{n}-data-type**: Column's data type.

### Tests

On DBT you can define tests to validate the models. There exist 2 types of tests:

- **Singular Data Tests**: Tests defined in the **tests** directory as a query that returns rows only if invalid data exists.
- **Generic Tests**: Test defined in the **tests/generic** able to be applied to any model and column in the **schema.yml**. Check dbt tests page for more information.

**Note**: The example project contains both singular data and generic tests.